

UNIVERSITY OF MIANWALI



ASSIGNMENT

Lab Task 1

SUBMITTED BY:

Name : Hina Bashir
Roll no : BCS_F24_Eo4
Semester : 5th self
Department : CS

SUBMITTED TO:

Name : Sir Raza
Subject : Computer Architecture
Department : CS

Task 1: Put a Number in a Register

```
global _main
extern _printf

section .data
; %d tells printf to display a decimal number.
; 10 creates a new line.
; 0 marks the end of the string.
format db "Value in AL = %d", 10, 0

section .text
_main:
; Put decimal 10 into AL.
; AL is an 8-bit register.
mov al, 10
; printf works with larger values.
; MOVZX copies AL into EAX and fills
; the remaining bits with zeros.
movzx eax, al
; Function parameters are pushed onto
; the stack from right to left.
push eax
; Push the address of the format string.
push format
; Call the printf function.
call _printf
; Two 32-bit values were pushed.
; Each value used 4 bytes.
; Remove 8 bytes from the stack.
add esp, 8
```

; Return 0 from the program.

xor eax, eax

ret

Code 2:

global _main

extern _printf

section .data

; %08X displays an 8-digit hexadecimal value.

format db "EAX = 0x%08X", 10, 0

section .text

_main:

; EAX is a 32-bit CPU register.

; Store hexadecimal 12345678 inside it.

mov eax, 0x12345678

; Push the number that printf will display.

push eax

; Push the address of the format string.

push format

; Display the result.

call _printf

; Remove the two parameters from the stack.

add esp, 8

; Return 0.

xor eax, eax

ret

Code 3 : Copy Data Between Registers

global _main

extern _printf

```

section .data
format db "AL = %d, BL = %d", 10, 0
section .text
_main:
; Store 10 inside AL.
mov al, 10
; Copy AL into BL.
mov bl, al
; Convert BL to a 32-bit value.
; Store it temporarily in ECX.
movzx ecx, bl
; Convert AL to a 32-bit value.
; Store it in EAX.
movzx eax, al
; printf arguments are pushed
; from right to left.
push ecx
push eax
push format
call _printf
; Three values were pushed.
; 3 x 4 = 12 bytes.
add esp, 12
xor eax, eax
ret

```

Code 4 : Add a Number

```

global _main
extern _printf

```

```

section .data
format db "10 + 5 = %d", 10, 0
section .text
_main:
; Start with 10.
mov al, 10
; Add 5 to AL.
add al, 5
; AL now contains 15.
; Convert the 8-bit result to 32 bits.
movzx eax, al
push eax
push format
call _printf
add esp, 8
xor eax, eax
ret

```

Code 5 : Add Two Registers

```

global _main
extern _printf
section .data
format db "AL = %d, BL = %d", 10, 0
section .text
_main:
; Store 10 in AL.
mov al, 10
; Store 20 in BL.
mov bl, 20

```

; Add the contents of BL to AL.

add al, bl

; AL now contains 30.

; BL still contains 20.

movzx ecx, bl

movzx eax, al

push ecx

push eax

push format

call _printf

add esp, 12

xor eax, eax

ret

Code 6 : Perform 32-Bit Addition

global _main

extern _printf

section .data

format db "EAX = %d, EBX = %d", 10, 0

section .text

_main:

; Store 10 inside EAX.

mov eax, 10

; Store 20 inside EBX.

mov ebx, 20

; Add EBX to EAX.

add eax, ebx

; EAX now contains 30.

; EBX still contains 20.

```
push ebx
push eax
push format
call _printf
add esp, 12
xor eax, eax
ret
```

Code 7 : Basic Subtraction

```
global _main
extern _printf
section .data
format db "20 - 5 = %d", 10, 0
section .text
_main:
; Store 20 inside AL.
mov al, 20
; Subtract 5.
sub al, 5
; AL now contains 15.
movzx eax, al
push eax
push format
call _printf
add esp, 8
xor eax, eax
ret
```

Code 8 : Subtract One Register From Another

```
global _main
```

```

extern _printf
section .data
format db "30 - 10 = %d", 10, 0
section .text
_main:
mov eax, 30
mov ebx, 10
; Calculate EAX - EBX.
sub eax, ebx
; EAX now contains 20.
push eax
push format
call _printf
add esp, 8
xor eax, eax
ret

```

Code 9 : Increment a Register

```

global _main
extern _printf
section .data
format db "After INC, EAX = %d", 10, 0
section .text
_main:
; Start with 9.
mov eax, 9
; INC means increase by one.
inc eax
; EAX now contains 10.

```

```
push eax
push format
call _printf
add esp, 8
xor eax, eax
ret
```

Code 10 : Decrement a Register

```
global _main
extern _printf
section .data
format db "After DEC, EBX = %d", 10, 0
section .text
_main:
; Start with 10.
mov ebx, 10
; DEC means decrease by one.
dec ebx
; EBX now contains 9.
push ebx
push format
call _printf
add esp, 8
xor eax, eax
ret
```

Code 11 : Perform an AND Operation

```
global _main
extern _printf
section .data
```

```
format db "Result = 0x%02X", 10, 0
```

```
section .text
```

```
__main:
```

```
; AA in binary is:
```

```
; 10101010
```

```
mov al, 0xAA
```

```
; 0F in binary is:
```

```
; 00001111
```

```
and al, 0x0F
```

```
; Result:
```

```
; 00001010
```

```
; This is hexadecimal 0A.
```

```
movzx eax, al
```

```
push eax
```

```
push format
```

```
call _printf
```

```
add esp, 8
```

```
xor eax, eax
```

```
ret
```

Code 12 : Perform an OR Operation

```
global __main
```

```
extern _printf
```

```
section .data
```

```
format db "Result = %d", 10, 0
```

```
section .text
```

```
__main:
```

```
; Decimal 5:
```

```
; 00000101
```

```
mov al, 5
; Decimal 8:
; 00001000
or al, 8
; Result:
; 00001101
; Decimal 13.
movzx eax, al
push eax
push format
call _printf
add esp, 8
xor eax, eax
ret
```

Code 13 : Compare two Equal Numbers

```
global _main
extern _printf
section .data
equalMessage db "The numbers are equal.", 10, 0
notEqualMessage db "The numbers are not equal.", 10, 0
section .text
_main:
mov al, 10
mov bl, 10
; CMP internally performs:
; AL - BL
;
; The result is not stored.
```

; CPU flags are updated instead.

cmp al, bl

; JE means Jump if Equal.

; It checks the Zero Flag.

je numbers_are_equal

numbers_are_not_equal:

push notEqualMessage

call _printf

add esp, 4

jmp finished

numbers_are_equal:

push equalMessage

call _printf

add esp, 4

finished:

xor eax, eax

ret

Code 14 : Test Two Bit Patterns

global _main

extern _printf

section .data

zeroMessage db "No common 1 bits were found.", 10, 0

commonMessage db "Common 1 bits were found.", 10, 0

section .text

_main:

; AL = 11110000

mov al, 0xF0

; BL = 00001111

```

mov bl, 0x0F
; TEST performs a bitwise AND
; but does not save the answer.
; It only changes CPU flags.
test al, bl
; Jump if the result was zero.
jz no_common_bits
common_bits:
push commonMessage
call _printf
add esp, 4
jmp finished
no_common_bits:
push zeroMessage
call _printf
add esp, 4
finished:
xor eax, eax
ret

```

Code 15 : Exchange Two Registers

```

global _main
extern _printf
section .data
format db "AL = %d, BL = %d", 10, 0
section .text
_main:
mov al, 10
mov bl, 20

```

```
; Before XCHG:
```

```
;
```

```
; AL = 10
```

```
; BL = 20
```

```
xchg al, bl
```

```
; After XCHG:
```

```
;
```

```
; AL = 20
```

```
; BL = 10
```

```
movzx ecx, bl
```

```
movzx eax, al
```

```
push ecx
```

```
push eax
```

```
push format
```

```
call _printf
```

```
add esp, 12
```

```
xor eax, eax
```

```
ret
```

Code 16 : Store a Number in Memory

```
global _main
```

```
extern _printf
```

```
section .data
```

```
; Reserve one byte in memory.
```

```
; Its starting value is 0.
```

```
number db 0
```

```
format db "Value in memory = %d", 10, 0
```

```
section .text
```

```
_main:
```

```
; Store decimal 42 at the memory
; location called number.
mov byte [number], 42
; Read the byte from memory and
; convert it to a 32-bit value.
movzx eax, byte [number]
push eax
push format
call _printf
add esp, 8
xor eax, eax
ret
```

Code 17 : Read Data From Memory

```
global _main
extern _printf
section .data
number db 0
format db "Value loaded into BL = %d", 10, 0
section .text
_main:
; Store 42 in memory.
mov byte [number], 42
; Read the byte from memory
; and place it inside BL.
mov bl, [number]
; Convert BL to 32 bits for printf.
movzx eax, bl
push eax
```

```
push format
call _printf
add esp, 8
xor eax, eax
ret
```

Code 18 : Add a Register Value to Memory

```
global _main
extern _printf
section .data
; Create one byte containing 10.
number db 10
format db "New memory value = %d", 10, 0
section .text
_main:
; Put 5 into BL.
mov bl, 5
; Add BL to the byte stored in memory.
;
; Memory initially contains 10.
; BL contains 5.
;
; 10 + 5 = 15
add byte [number], bl
; Read the new memory value.
movzx eax, byte [number]
push eax
push format
call _printf
```

```
add esp, 8
xor eax, eax
ret
```

Code 19 : Use the Stack

```
section .data
format db "EAX = 0x%08X, EBX = 0x%08X", 10, 0
section .text
_main:
; Put a value into EAX.
mov eax, 0x12345678
; PUSH places EAX on the stack.
push eax
; POP removes the top value
; from the stack and places it
; inside EBX.
pop ebx
; Both registers now contain
; the same value.
push ebx
push eax
push format
call _printf
add esp, 12
xor eax, eax
ret
```

Code 20 : Mini Complete Program

```
global _main
extern _printf
```

```
section .data
; Reserve one byte for the final result.
result db 0
; printf will display three decimal numbers.
format db "AL = %d, BL = %d, Memory = %d", 10, 0
section .text
__main:
; -----
; Step 1
; Put 10 into AL.
; -----
mov al, 10
; -----
; Step 2
; Add 20.
;
; 10 + 20 = 30
; -----
add al, 20
; -----
; Step 3
; Subtract 5.
;
; 30 - 5 = 25
; -----
sub al, 5
; -----
; Step 4
```

```
; Store AL in memory.
;
; Memory now contains 25.
; -----
mov [result], al
; -----
; Step 5
; Read the value from memory into BL.
; -----
mov bl, [result]
; -----
; Prepare values for printf.
;
; printf uses 32-bit integer parameters.
; -----
movzx eax, al
movzx ecx, bl
movzx edx, byte [result]
; -----
; printf arguments must be pushed
; from right to left.
;
; format contains:
;
; AL = %d, BL = %d, Memory = %d
;
; Therefore we push:
;
```

address

; -----

push edx

push ecx

push eax

push format

call _printf

; -----

; Four 32-bit parameters were pushed.

;

; 4 x 4 bytes = 16 bytes.

;

; Restore ESP.

; -----

add esp, 16

; -----

; Return 0 to Windows.

; -----

xor eax, eax

ret